



操作系统实验报告

实验三：调度算法比较

专 业： 人工智能

姓 名： 张晓宇

学 号： 37220232203927

实验日期： 2025.12.5

指导教师： 曹冬林

2025 年 12 月 24 日

目录

1	实验目的	3
1.1	深入理解进程调度机制	3
1.2	掌握多种调度算法的原理与实现	3
1.3	设计调度算法评估框架	3
1.4	掌握调度算法性能分析方法	3
2	实验环境	3
2.1	硬件环境	3
2.2	软件环境	4
3	实验要求	4
3.1	任务一：实现调度算法	4
3.2	任务二：实现测试程序	4
4	实验流程	4
4.1	实验准备：调度框架设计	5
4.1.1	扩展进程控制块	5
4.1.2	调度框架设计	6
4.1.3	就绪队列实现	6
4.1.4	进程统计信息收集	8
4.2	任务一：实现调度算法	8
4.2.1	调度器主循环	8
4.2.2	优先级调度算法（Priority Scheduling）	11
4.2.3	先来先服务调度算法（FCFS）	13
4.2.4	轮转调度算法（Round Robin）	14
4.2.5	静态多级队列调度算法（SML）	15
4.3	任务二：实现测试程序	17
4.3.1	测试程序思路	17
4.3.2	wait2 系统调用	19
4.4	测试与验证	20
4.4.1	场景 1：车队效应分析	21
4.4.2	场景 2：交互响应分析	22
4.4.3	场景 3：优先级饥饿分析	22
4.4.4	运行结果分析	23

5	实验总结	24
5.1	掌握了调度框架的设计方法	24
5.2	深入理解了不同调度算法的特点	25
5.3	掌握了数据结构优化技术	25
5.4	掌握了进程统计信息收集方法	25
5.5	理解了调度决策对系统性能的影响	25
6	实验心得	25
7	附录：核心代码实现	27
7.1	附录 A：数据结构定义	27
7.1.1	A.1 ptable.h - 进程信息结构	27
7.1.2	A.2 proc.h - 进程控制块扩展	28
7.1.3	A.3 proc.c - 信号量结构定义	29
7.2	附录 B：进程管理系统调用实现	29
7.2.1	B.1 proc.c - setpriority 函数	29
7.2.2	B.2 proc.c - getptable 函数	30
7.2.3	B.3 sysproc.c - 系统调用包装器	32
7.3	附录 C：用户态程序实现	33
7.3.1	C.1 ps.c - 进程查看命令	33
7.3.2	C.2 nice.c - 优先级修改命令	35
7.4	附录 D：信号量实现	36
7.4.1	D.1 proc.c - sem_init 函数	36
7.4.2	D.2 proc.c - sem_destroy 函数	37
7.4.3	D.3 proc.c - sem_wait 函数 (P 操作)	38
7.4.4	D.4 proc.c - sem_signal 函数 (V 操作)	39
7.4.5	D.5 sysproc.c - 信号量系统调用包装器	40
7.5	附录 E：信号量测试程序	42
7.5.1	E.1 sem_test.c - 完整实现	42

1 实验目的

本次实验围绕操作系统的核心主题——进程调度算法展开，旨在达成以下目标：

1.1 深入理解进程调度机制

学习操作系统中进程调度器的工作原理，理解调度器如何在多个就绪进程之间进行选择。掌握调度策略与调度算法的区别，以及不同调度算法对系统性能的影响。

1.2 掌握多种调度算法的原理与实现

研究并实现四种经典的进程调度算法：

- **优先级调度 (Priority Scheduling)**：根据进程优先级选择执行，优先级高的进程优先获得 CPU。
- **先来先服务 (FCFS)**：按照进程到达就绪队列的先后顺序进行调度。
- **轮转调度 (Round Robin)**：进程按照循环顺序轮流获得时间片执行。
- **静态多级队列 (SML)**：将进程按优先级划分到不同队列，高优先级队列优先调度，队列内部使用轮转策略。

1.3 设计调度算法评估框架

构建一个可切换的调度框架，支持在运行时动态切换调度算法。设计合理的测试方案，包含不同类型的负载（CPU 密集型和 IO 密集型），对调度算法进行公平的性能对比。

1.4 掌握调度算法性能分析方法

学习进程运行时间统计方法，理解就绪时间、运行时间、休眠时间、周转时间等关键指标的含义。通过对比实验，分析不同调度算法在各项指标上的表现差异。

2 实验环境

2.1 硬件环境

- CPU：Intel i9-13900h
- GPU：NVIDIA RTX 4050
- 宿主机操作系统：Windows 11

2.2 软件环境

- 操作系统: Ubuntu 18.04 (64 位)
- 模拟器: QEMU
- 编译工具: gcc, make (build-essential)
- 目标操作系统内核: xv6

3 实验要求

3.1 任务一：实现调度算法

在 `proc.c` 中编程实现以下四种调度算法：

1. 优先级调度 (Priority Scheduling)：选择优先级最高（数值最小）的就绪进程执行。
2. 先来先服务 (FCFS)：选择创建时间最早的就绪进程执行。
3. 轮转调度 (Round Robin)：按循环顺序公平地调度每个就绪进程。
4. 静态多级队列 (SML)：按优先级将进程划分为多个队列，高优先级队列优先执行。

3.2 任务二：实现测试程序

在 `scheduler_test.c` 中实现测试程序：

1. 创建 6 个进程，包含 3 个 CPU 密集型进程和 3 个 IO 密集型进程。
2. 为不同类型的进程设置不同的优先级。
3. 记录所有进程的运行过程，收集进程统计信息。
4. 对比各调度算法在平均就绪时间、平均运行时间、平均休眠时间、平均周转时间上的结果。

4 实验流程

实验三完成思路流程图如下：

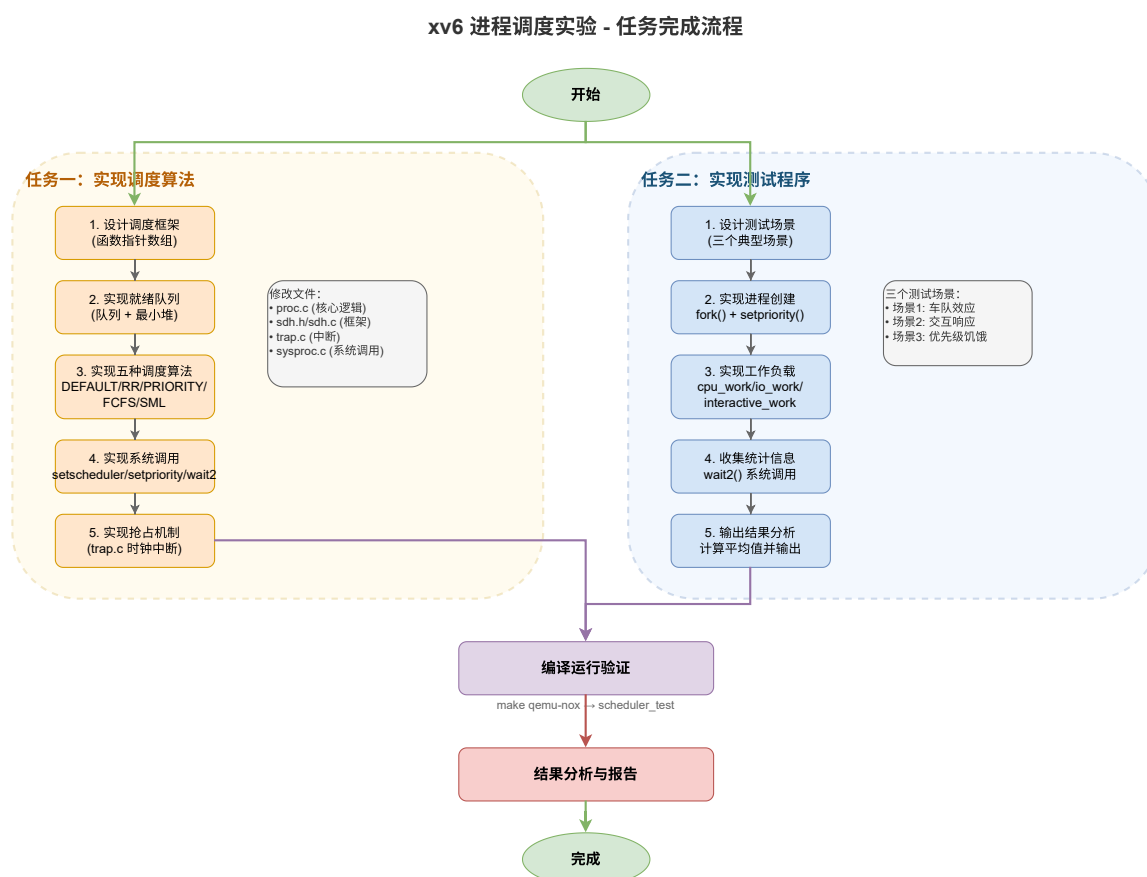


图 1 实验完成流程

4.1 实验准备：调度框架设计

本次实验需要设计一个可扩展的调度框架，支持在运行时动态切换调度算法。同时需要扩展进程控制块以支持运行时间统计。

4.1.1 扩展进程控制块

为了支持调度算法比较，需要在 `proc.h` 的进程控制块中添加以下时间统计字段：

- `priority`: 进程优先级 (1-20)
- `ctime`: 进程创建时间
- `stime`: 休眠 (SLEEPING) 累计时间
- `retime`: 就绪 (RUNNABLE) 累计时间
- `runtime`: 运行 (RUNNING) 累计时间

4.1.2 调度框架设计

在 `sdh.h` 中设计可扩展的调度框架，核心思想是使用函数指针数组存储各调度算法，通过 `setscheduler()` 系统调用动态切换当前使用的调度函数。

调度框架的设计思路如图??所示。

4.1.3 就绪队列实现

为了提高调度效率，实现了真正的就绪队列数据结构。

队列与堆结构设计：

在 `proc.c` 中定义了不同的数据结构支持各种调度算法：

- `fcfsQueue`：用于 FCFS 和 RR 调度的先进先出队列（1 个队列）
- `pHeap`：用于优先级调度的最小堆结构，时间复杂度 $O(\log n)$
- `smlQueues[3]`：用于静态多级队列调度（3 个队列：高/中/低）

最小堆优化说明：

优先级调度采用最小堆而非多级队列实现，具有以下优势：

- 插入进程： $O(\log n)$ ，通过 `siftUp()` 上浮操作维护堆性质
- 提取最高优先级进程： $O(\log n)$ ，通过 `siftDown()` 下沉操作
- 堆的性质：父节点优先级 $\leq$ 子节点优先级（数值越小优先级越高）

核心实现：

```
enqueue

1 // 入队：将进程添加到队列尾部
2 void enqueue(struct procqueue *q, struct proc *p) {
3     p->next = 0;
4     if (q->tail == 0) {
5         q->head = q->tail = p;
6     } else {
7         q->tail->next = p;
8         q->tail = p;
9     }
10 }
```

```
dequeue

1 // 出队：从队列头部取出进程
2 struct proc *dequeue(struct procqueue *q) {
3     struct proc *p = q->head;
4     if (p == 0)
5         return 0;
6     q->head = p->next;
7     if (q->head == 0) {
8         q->tail = 0;
9     }
10    p->next = 0;
11    return p;
12 }
```

```
addToReadyQueues

1
2 // 统一入队函数：进程变为 RUNNABLE 时调用
3 // 根据当前激活的调度器，只添加到对应的队列
4 extern int schedSelected; // 来自 sdh.h，表示当前调度器
   类型
5
6 void addToReadyQueues(struct proc *p) { // 将进程 p 加入到
   就绪队列
7     switch (schedSelected) {
8         case 0: // DEFAULT - 使用 fcfsQueue
9         case 2: // FCFS
10        case 3: // RR (CFS)
11            enqueue(&fcfsQueue, p);
12            break;
13        case 1: // PRIORITY - 使用最小堆
14            heapInsert(p);
15            break;
16        case 4: // SML
17            if (p->priority >= 1 && p->priority <= 7) {
18                enqueue(&smlQueues[0], p);
```

```
addToReadyQueues
19         } else if (p->priority >= 8 && p->priority <= 14)
20         {
21             enqueue(&smlQueues[1], p);
22         } else {
23             enqueue(&smlQueues[2], p);
24         }
25         break;
26     default:
27         enqueue(&fcfsQueue, p);
28         break;
29     }
```

调度器选择机制：

调度器选择通过编译时宏定义和运行时函数指针两种机制实现：

1. **编译时：**通过 Makefile 传递 SCHEDPOLICY 宏，在 `sdh.h` 中初始化 `ready_process` 函数指针
2. **运行时：**通过 `setscheduler(sid)` 系统调用动态切换调度器，同时重建就绪队列

调度器选择机制流程如图1所示。

4.1.4 进程统计信息收集

实现 `update_statistics()` 函数，在每个时钟中断时被调用，根据当前进程状态更新对应的时间计数器。该函数遍历进程表，根据进程的 SLEEPING、RUNNABLE、RUNNING 状态分别递增 `stime`、`retime`、`runtime` 计数器。

4.2 任务一：实现调度算法

4.2.1 调度器主循环

修改 `scheduler()` 函数，使用函数指针 `ready_process` 调用当前选择的调度算法，实现调度策略的动态切换。

调度器函数：

```
1 // scheduler - 调度器主循环函数
2 // 功能：操作系统的核心调度循环，负责选择并切换到下一个要执行的进程
3 // 说明：此函数永不返回，每个CPU核心启动后会一直在此循环中运行
4 void scheduler(void) {
5     struct proc *p;          // 指向被选中进程的指针
6     struct cpu *c = mycpu(); // 获取当前CPU结构体
7     c->proc = 0;             // 初始化：当前CPU没有运行任何进程
8
9     for (;;) { // 无限循环：调度器永不停止
10        // 步骤1：启用中断
11        sti(); // Set Interrupt flag (设置中断标志) 主要目的：
12        // 若无就绪进程，CPU会空转，设置中断标志后时钟中断可以触发，从而调用wakeup()。否则会死锁
13
14        // 步骤2：获取进程表锁，准备查找就绪进程
15        acquire(&ptable.lock);
16
17        // 步骤3：调用当前选定的调度算法选择下一个进程
18        acquire(&schedulerlock);
19        p = (*ready_process)(); // 调用调度算法函数（如priorityScheduler、fcfsScheduler等）
20        release(&schedulerlock);
21
22        // 步骤4：如果找到了就绪进程，则进行上下文切换
23        if (p != 0) {
24            // 4.1 设置当前CPU正在运行的进程
25            c->proc = p;
26
27            // 4.2 切换到进程的页表（用户地址空间）
28            switchvm(p);
29
30            // 4.3 将进程状态标记为RUNNING
31            p->state = RUNNING;
32
```

```
33 // 4.4 执行上下文切换：保存调度器上下文，恢复进程上下文
34 swtch(&(c->scheduler), p->context);
35
36 // 4.5 进程p切换回来后，恢复内核页表
37 switchkvm();
38
39 // 4.6 清理：当前CPU不再运行任何进程
40 c->proc = 0;
41 }
42
43 // 步骤5：释放进程表锁
44 release(&ptable.lock);
45 }
46 }
```

调度器主循环流程如图2所示。

scheduler() 调度器主循环流程

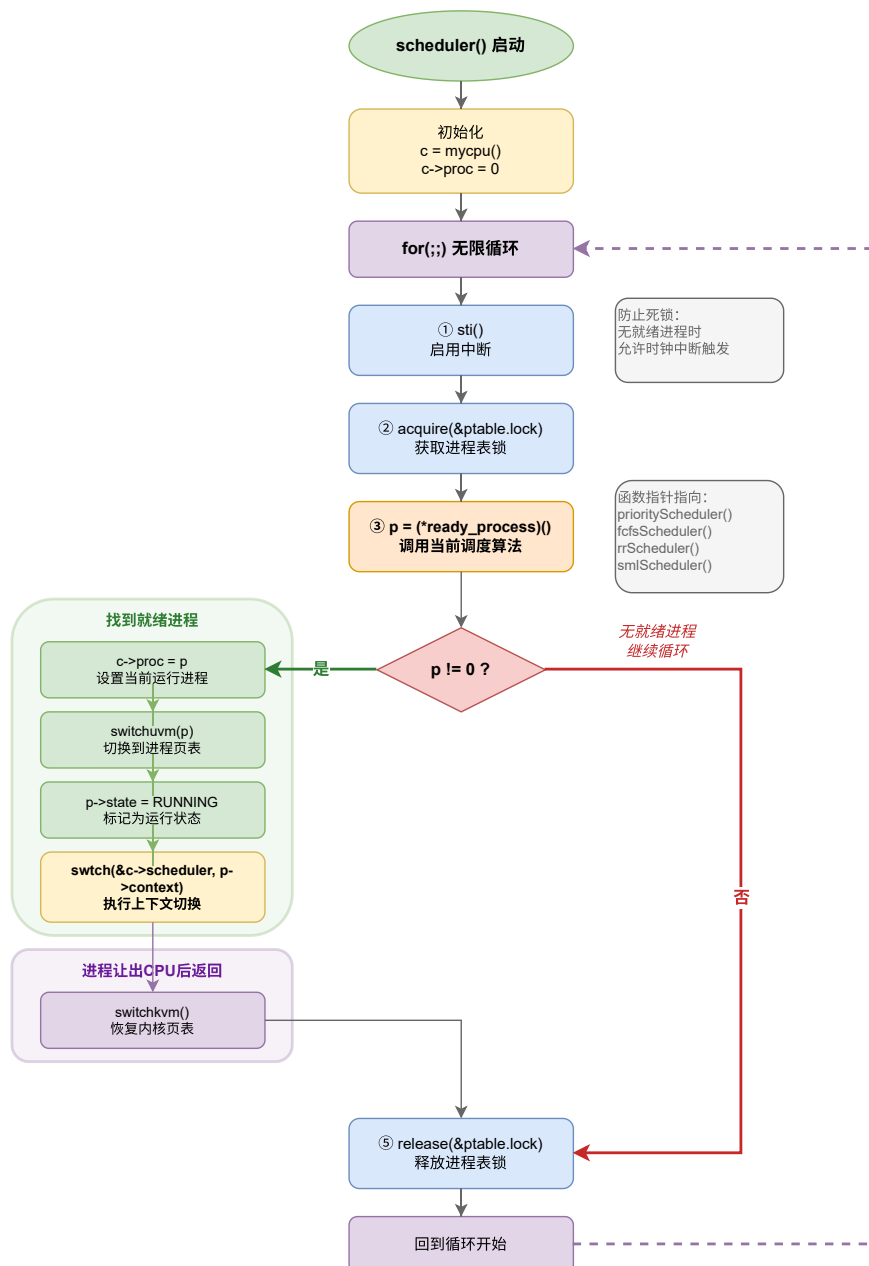


图 2 调度器主循环流程

4.2.2 优先级调度算法 (Priority Scheduling)

算法原理:

优先级调度是一种基于进程重要性的抢占式或非抢占式调度算法。每个进程被分配一个优先级值，调度器总是选择优先级最高的就绪进程执行。在本实现中，优先级使用

数值 1-20 表示，数值越小表示优先级越高。

该算法的核心思想是：重要的任务应该优先得到 CPU 资源。通过为不同类型的进程分配不同的优先级，系统可以确保关键任务（如系统服务、实时任务）能够快速响应，而次要任务（如后台批处理）则在资源空闲时执行。

实现策略：

本实现采用**最小堆**数据结构实现非抢占式优先级调度，具体设计如下：

1. 使用结构体 `priorityHeap` 存储就绪进程指针数组和堆大小
2. 插入时调用 `heapInsert()`，将进程放入堆末尾后执行 `siftUp()` 上浮
3. 提取时调用 `heapExtract()`，取出堆顶后将末尾元素放到堆顶，执行 `siftDown()` 下沉
4. 堆顶始终是优先级最高（数值最小）的进程

相比遍历进程表的 $O(n)$ 实现，最小堆将提取操作优化到 $O(\log n)$ 。

1. 遍历整个进程表，筛选出所有状态为 `RUNNABLE` 的就绪进程
2. 维护一个候选进程指针 `highP`，初始值为 `NULL`
3. 对每个就绪进程，比较其 `priority` 字段与当前候选进程
4. 如果发现更高优先级的进程（数值更小），则更新候选进程指针
5. 遍历完成后，返回优先级最高的进程

算法优势：

- **响应快速**：高优先级进程可以立即获得 CPU，适合实时系统
- **灵活性高**：可以根据实际需求动态调整进程优先级
- **实现简单**：只需一次遍历即可完成调度决策

潜在问题：

- **饥饿问题**：低优先级进程可能永远得不到执行机会
- **不够公平**：相同优先级的进程之间没有轮转机制

优先级调度算法的实现流程如图??所示。

核心代码实现：

优先级调度算法的核心代码如下。

```
priorityScheduler

1 // Priority Scheduler -----
2 // 使用最小堆提取优先级最高（数值最小）的 RUNNABLE 进程
3 // 时间复杂度  $O(\log n)$ 
4 struct proc *priorityScheduler() {
5     return heapExtract(); // 从最小堆中提取堆顶（优先级最高
6                             的进程）
7 }
```

4.2.3 先来先服务调度算法（FCFS）

算法原理：

先来先服务（First-Come First-Served）是最简单直观的调度算法，遵循“先到先得”的公平原则。进程按照进入就绪队列的时间顺序排队，调度器总是选择最早到达的进程执行，直到该进程执行完毕或主动放弃 CPU。

实现策略：

```
fcfsScheduler

1 // FCFS Scheduler -----
2 struct proc *fcfsScheduler() { // 先来先服务
3     return dequeue(&fcfsQueue); // 从就绪顺序队列中取出
4 }
```

算法优势：

- 实现简单：无需复杂的数据结构和调度决策逻辑
- 公平性好：所有进程按到达顺序依次执行，不会饥饿
- 可预测性强：进程等待时间可以根据队列长度估算

潜在问题：

- 护航效应（Convoy Effect）：一个长进程会阻塞后面所有短进程
- 平均等待时间长：短作业可能因为前面的长作业而等待很久
- 不适合交互式系统：无法优先响应用户交互请求

FCFS 调度算法原理如图3所示。

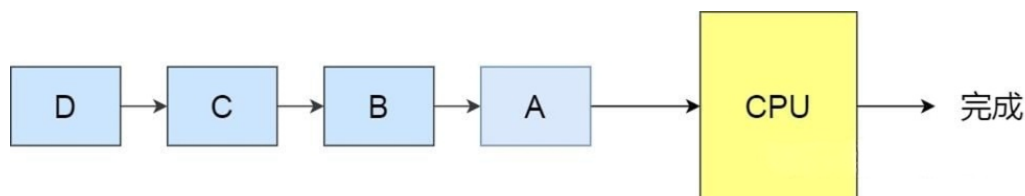


图 3 先来先服务调度原理

4.2.4 轮转调度算法 (Round Robin)

算法原理：

轮转调度 (Round Robin, 简称 RR) 是一种时间片轮转的抢占式调度算法, 被广泛应用于分时操作系统中。其核心思想是: 将 CPU 时间划分为固定长度的时间片 (time quantum), 就绪队列中的进程按循环顺序依次获得一个时间片的执行机会。

当一个进程的时间片用完后, 即使它尚未完成, 也会被迫让出 CPU, 回到就绪队列末尾等待下一轮调度。这种机制保证了所有进程都能公平地获得 CPU 资源, 不会出现某个进程长期占用 CPU 的情况。

实现策略：

```
rrScheduler

1 // Round Robin Scheduler -----
2 struct proc *rrScheduler() { // 轮转调度
3     return dequeue(&fcfsQueue); // 从FCFS队列取出, 调度后由
   yield重新入队尾
4     // 当前文件中与先来先服务的实现相同, 但是在trap.c中, 只
   有轮转是抢占式, 即只有轮转会被时钟中断抢占
5 }
```

时间片设置：在 xv6 中, 时间片由时钟中断频率决定, 默认为 10ms。只有 RR 调度器在时钟中断时会强制调用 `yield()` 让出 CPU, 其他调度器 (FCFS、PRIORITY、SML) 均为非抢占式。

算法优势：

- 公平性极佳: 所有进程获得相同的 CPU 时间配额
- 响应时间好: 交互式任务不会等待太久
- 无饥饿问题: 每个进程都保证在有限时间内得到调度
- 适合分时系统: 特别适合多用户、多任务环境

潜在问题：

- 上下文切换开销：频繁切换进程会增加系统开销
- 时间片难以选择：过长影响响应时间，过短增加开销
- 不区分优先级：无法优先服务重要进程

轮转调度算法原理如图4所示。

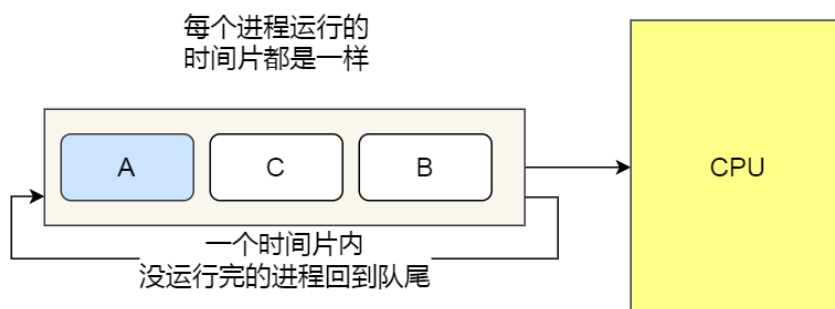


图 4 轮转调度原理

4.2.5 静态多级队列调度算法（SML）

算法原理：

静态多级队列（Static Multi-Level Queue）是一种混合型调度算法，它巧妙地结合了优先级调度和轮转调度的优点。该算法的核心思想是：根据进程的优先级将其永久性地分配到不同的就绪队列中，高优先级队列总是优先得到调度，而队列内部则采用轮转策略保证公平性。

”静态”体现在进程一旦被分配到某个队列，其所属队列关系就固定不变；”多级”则体现在系统维护多个不同优先级的就绪队列。这种设计既保证了重要进程的响应速度，又避免了同等重要进程之间的不公平竞争。

队列划分策略：

本实现将进程按优先级划分为三个队列：

- 队列 1（高优先级）：priority 1-7 （系统服务、实时任务）
- 队列 2（中优先级）：priority 8-14 （普通用户进程）
- 队列 3（低优先级）：priority 15-20 （后台批处理）

静态多级队列调度算法原理如图5所示。

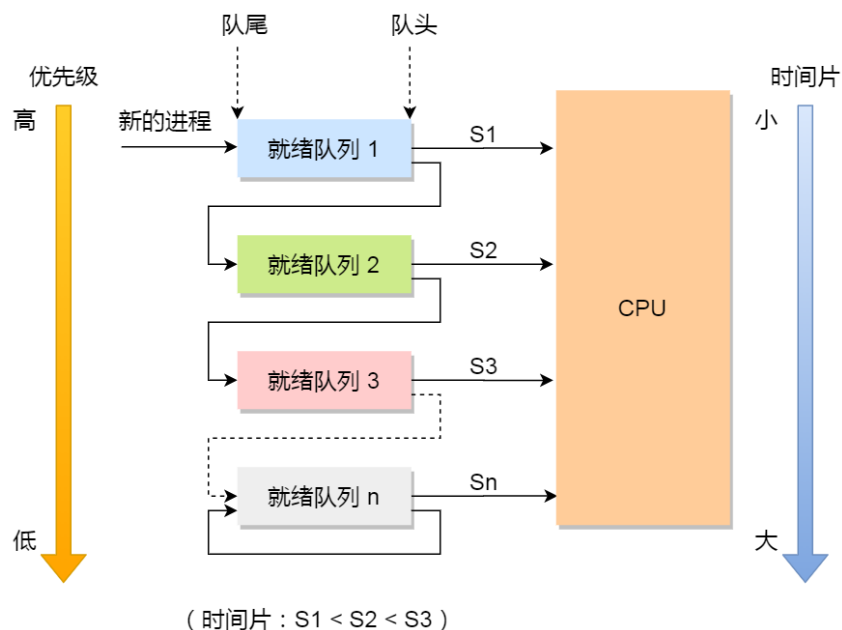


图 5 静态多级队列调度原理

实现策略：

调度器按照以下严格顺序选择进程：

1. 优先调度队列 1：使用轮转方式从高优先级队列中选择进程
2. 如果队列 1 为空，则调度队列 2：同样使用轮转策略
3. 如果队列 2 也为空，才调度队列 3：继续使用轮转策略
4. 每个队列维护独立的 `lastIdx` 索引，记录上次调度位置
5. 当高优先级队列中有新进程就绪时，立即抢占低优先级队列的 CPU

算法优势：

- 兼顾效率与公平：高优先级进程快速响应，同级进程公平竞争
- 调度开销较低：只需遍历当前优先级队列，不必扫描全部进程
- 易于实现：逻辑清晰，代码结构简单
- 灵活可扩展：可根据需要增加或减少队列数量

潜在问题：

- 低优先级饥饿：如果高优先级队列持续有进程，低优先级进程可能长期得不到调度
- 队列边界敏感：优先级 7 和 8 的进程可能表现差异很大

- 缺乏动态调整：无法根据进程行为自动提升或降低优先级

改进空间：可以引入老化机制（aging），当低优先级进程等待时间过长时，临时提升其优先级，避免饥饿问题。

算法实现如下图所示。

```
smlScheduler

1 // SML (Static Multi-Level Queue) Scheduler
2 -----
3 // 依次从高、中、低优先级队列出队，实现分层调度
4 // 高优先级队列优先，同级别内 FCFS
5 // 时间复杂度  $O(1)$ 
6 struct proc *smlScheduler() {
7     struct proc *p;
8
9     // 优先从高优先级队列出队
10    p = dequeue(&smlQueues[0]);
11    if (p != 0) return p;
12
13    // 高优先级队列空，尝试中优先级队列
14    p = dequeue(&smlQueues[1]);
15    if (p != 0) return p;
16
17    // 中优先级队列也空，尝试低优先级队列
18    return dequeue(&smlQueues[2]);
19 }
```

4.3 任务二：实现测试程序

4.3.1 测试程序设计思路

测试程序实现思路流程图如下：

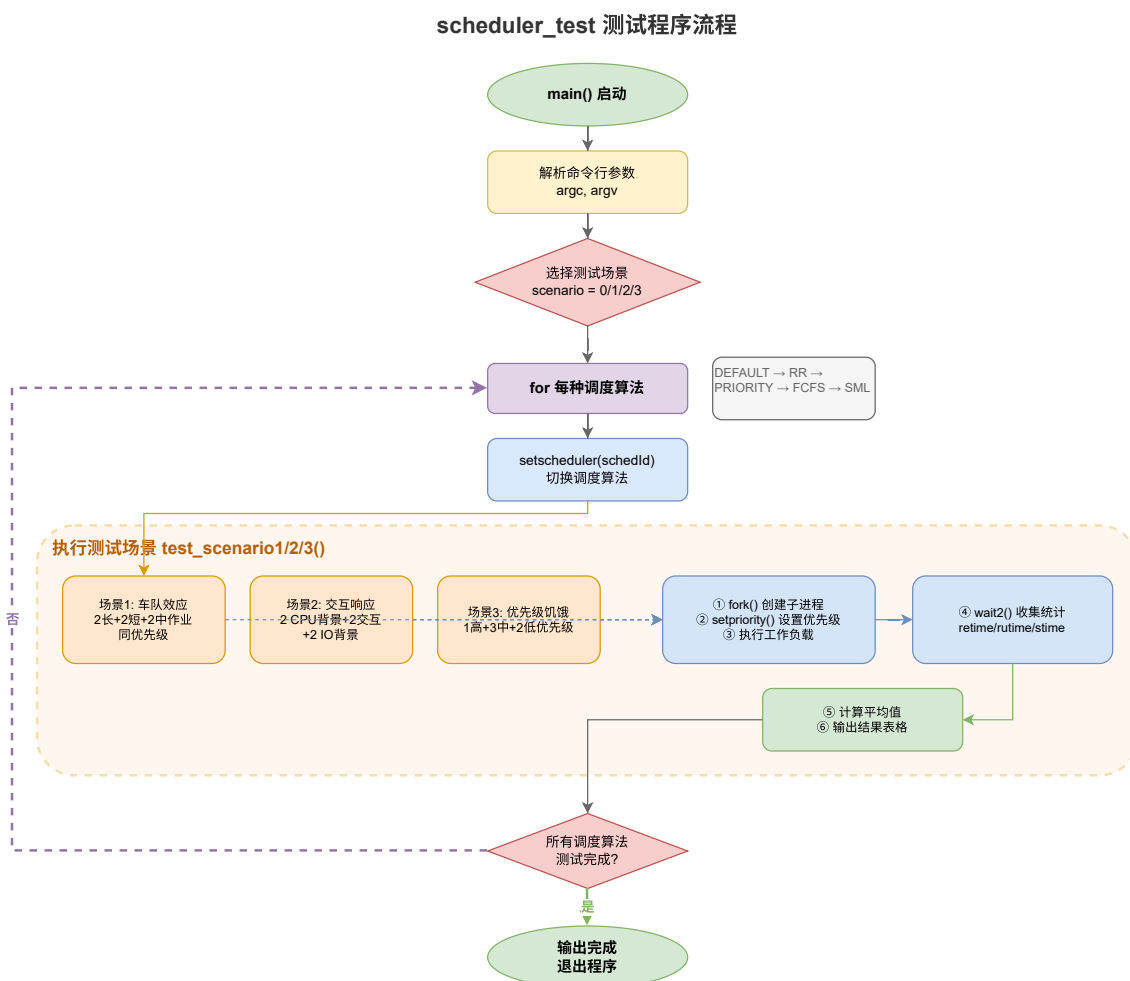


图 6 静态多级队列调度原理

为了全面对比不同调度算法的特性，测试程序 `scheduler_test.c` 设计了三个典型测试场景，分别针对调度算法的不同弱点进行测试：

场景 1：纯 CPU 长短作业（车队效应测试）

- 创建 2 个长作业（CPU_WORK）+ 2 个短作业（CPU_WORK/10）+ 2 个中等作业（CPU_WORK/3）
- 所有进程优先级相同（priority=10），排除优先级干扰
- 短作业比长作业晚 10 个时钟周期创建，模拟作业错峰到达
- 测试目的：展示 FCFS 的护航效应（Convoy Effect）以及 RR 的公平性优势

场景 2：交互型 vs 背景任务（响应性测试）

- 2 个 CPU 背景任务（低优先级 18，执行大量计算）

- 2 个交互型任务（高优先级 3，短 CPU 计算 + 频繁 sleep 模拟用户等待）
- 2 个 IO 背景任务（中优先级 12，文件 IO + sleep）
- 测试目的：展示 RR/SML 调度器对交互型进程的响应友好性

场景 3：优先级饥饿测试（公平性测试）

- 1 个超高优先级大任务（priority=1，CPU_WORK*2，长期霸占 CPU）
- 3 个中优先级小任务（priority=10）
- 2 个低优先级小任务（priority=19，可能发生饥饿）
- 测试目的：展示 Priority 调度的饥饿问题，以及 SML/RR 如何缓解

命令行参数支持：

测试程序支持通过命令行参数选择运行场景：

- scheduler_test —运行全部 3 个场景
- scheduler_test 1 —仅运行场景 1（车队效应）
- scheduler_test 2 —仅运行场景 2（交互响应）
- scheduler_test 3 —仅运行场景 3（优先级饥饿）

每个场景对 5 种调度算法进行对比测试：DEFAULT、RR（轮转）、PRIORITY、FCFS、SML（多级队列）。测试结果同时输出到屏幕和 sched_result.txt 文件。

4.3.2 wait2 系统调用

为了收集子进程的运行统计信息，实现 wait2() 系统调用。该系统调用在原有 wait() 的基础上，额外返回子进程的就绪时间、运行时间和休眠时间三个统计指标。

wait2 系统调用的实现流程如图7所示。

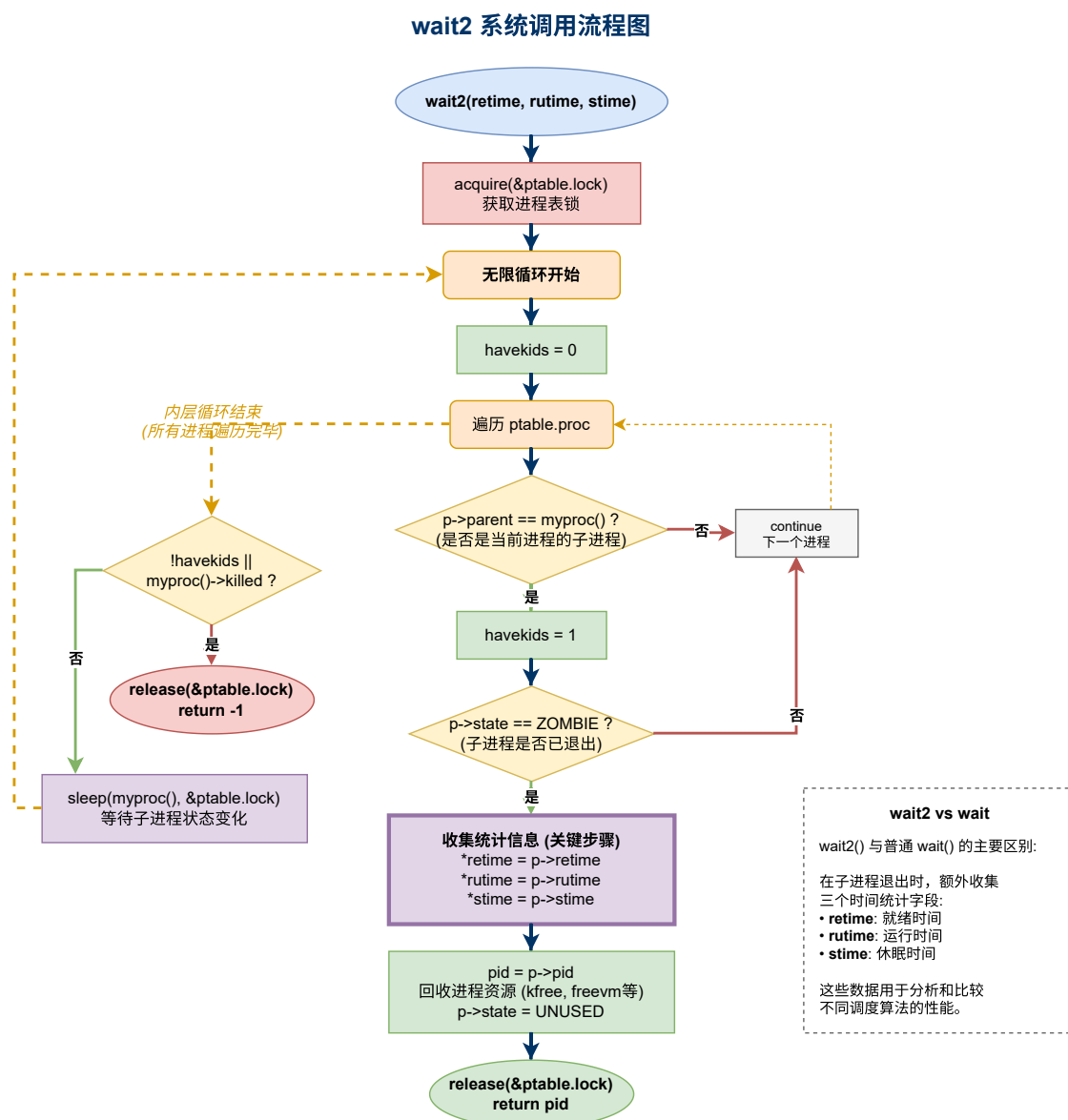


图 7 wait2 系统调用实现流程

4.4 测试与验证

编译并启动内核 `make qemu-nox`, 运行 `scheduler_test` 测试各调度算法。

测试配置:

测试程序按场景分别创建进程:

- 场景 1 (车队效应): 2 长作业 + 2 短作业 + 2 中作业, 同优先级
- 场景 2 (交互响应): 2 CPU 背景 + 2 交互型 + 2 IO 背景, 差异化优先级
- 场景 3 (优先级饥饿): 1 高优先级大任务 + 3 中优先级 + 2 低优先级

性能指标解释：

- 就绪时间 (Ready Time)：进程在就绪队列中等待 CPU 的时间
- 运行时间 (Run Time)：进程实际占用 CPU 执行的时间
- 休眠时间 (Sleep Time)：进程因等待 IO 等资源而睡眠的时间
- 周转时间 (Turnaround Time)：进程从创建到完成的总时间 (= 就绪 + 运行 + 休眠)

测试运行截图：

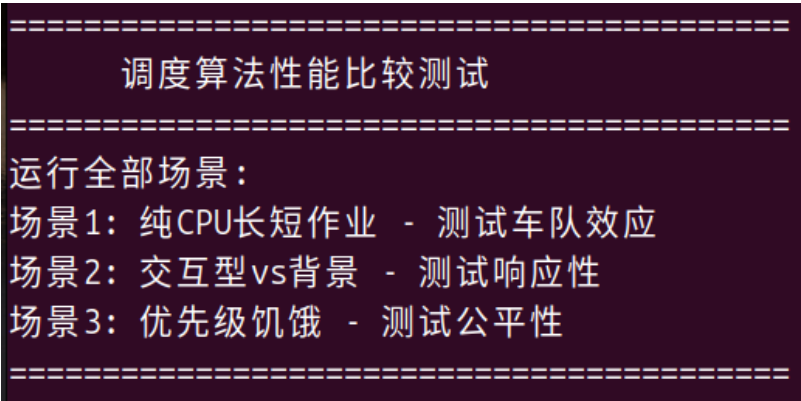


图 8 测试运行截图

4.4.1 场景 1：车队效应分析

预期行为：

- FCFS：长作业先执行完，短作业等待时间极长（护航效应）
- RR：长短作业交替执行，短作业周转时间显著减少
- PRIORITY/SML：同优先级下退化为类似 RR 的行为

测试运行截图：

===== DEFAULT - 场景1: 纯CPU长短作业(车队效应) =====					
PID	优先级	类型	就绪时间	运行时间	周转时间
----	-----	----	-----	-----	-----
4	10	长作业	21	220	241
5	10	长作业	256	218	474
6	10	短作业	459	23	513
7	10	短作业	473	22	565
8	10	中作业	478	77	609
9	10	中作业	543	76	621
----	-----	----	-----	-----	-----
长作业	平均		138	219	357
短作业	平均		466	22	488
中作业	平均		510	76	587

图 9 测试运行截图

4.4.2 场景 2：交互响应分析

预期行为：

- RR：交互型进程能快速获得响应，但不考虑优先级
- PRIORITY/SML：高优先级交互进程响应最快
- FCFS：交互进程可能被 CPU 背景任务阻塞

测试运行截图：

===== PRIORITY - 场景2：交互型vs背景任务 =====						
PID	优先级	类型	就绪时间	运行时间	休眠时间	周转时间
49	3	交互型	24	80	183	287
48	3	交互型	9	80	253	342
46	18	CPU背景	285	232	0	517
50	12	IO背景	248	43	366	657
47	18	CPU背景	477	244	0	721
51	12	IO背景	225	40	469	734

交互型	平均		16	80	218	314
CPU背景	平均		381	238	0	619
IO背景	平均		236	41	417	695

图 10 测试运行截图

4.4.3 场景 3：优先级饥饿分析

预期行为：

- PRIORITY：低优先级进程饥饿，等待高优先级大任务完成
- RR：忽略优先级，所有进程公平获得时间片
- SML：队列内轮转，一定程度缓解饥饿

测试运行截图：

===== PRIORITY - 场景3：优先级饥饿测试 =====					
PID	优先级	优先级组	就绪时间	运行时间	周转时间
52	1	高优先级	1	434	435
55	10	中优先级	561	109	712
53	10	中优先级	573	109	772
54	10	中优先级	656	116	772
56	19	低优先级	789	114	903
57	19	低优先级	885	112	997

高优	平均		1	434	435
中优	平均		596	111	708
低优	平均		837	113	950

图 11 测试运行截图

4.4.4 运行结果分析

场景 1：车队效应测试结果

表 1 场景 1 各调度算法平均性能对比（单位：ticks）

调度器	长作业			短作业			中作业		
	就绪	运行	周转	就绪	运行	周转	就绪	运行	周转
DEFAULT	142	220	363	465	22	488	510	75	586
RR	412	223	635	104	21	126	291	73	364
PRIORITY	219	219	438	22	22	44	74	73	147
FCFS	415	224	640	126	22	148	270	74	344
SML	415	224	639	107	22	129	295	74	369

关键发现：PRIORITY 调度器下短作业周转时间仅 44 ticks，远优于其他调度器。这是因为堆结构使短作业能够快速被提取执行。RR/FCFS/SML 的短作业周转时间也较短（126-148 ticks），表现出良好的公平性。

场景 2：交互响应测试结果

表 2 场景 2 各调度算法平均性能对比（单位：ticks）

调度器	交互型			CPU 背景			IO 背景		
	就绪	休眠	周转	就绪	运行	周转	就绪	休眠	周转
DEFAULT	485	157	723	123	220	343	472	357	876
RR	399	130	613	447	228	675	439	329	810
PRIORITY	16	229	325	365	239	604	225	495	763
FCFS	402	133	618	450	231	681	446	316	804
SML	76	197	355	491	240	732	236	409	688

关键发现：PRIORITY 下交互型进程就绪时间仅 16 ticks，周转时间 325 ticks；SML 下就绪时间 76 ticks，周转时间 355 ticks。两者都远优于其他调度器，体现了优先级调度对高优先级交互进程的快速响应能力。

场景 3：优先级饥饿测试结果

表 3 场景 3 各调度算法平均性能对比（单位：ticks）

调度器	高优先级		中优先级		低优先级	
	就绪	周转	就绪	周转	就绪	周转
DEFAULT	41	482	595	707	844	957
RR	547	1004	558	667	564	673
PRIORITY	1	440	367	479	207	324
FCFS	551	1010	558	667	573	683
SML	1	440	371	484	450	565

关键发现：

- PRIORITY/SML：高优先级进程就绪时间仅 1 ticks，响应极快
- PRIORITY：低优先级进程就绪时间仅 207 ticks，周转时间 324 ticks，表现出乎意料地好（可能因为堆的特性使得进程能较快被调度）
- RR/FCFS：完全忽略优先级，各优先级组就绪时间接近（约 550-570 ticks）

四、综合性能对比

表 4 五种调度算法综合性能评估

评估维度	DEFAULT	RR	PRIORITY	FCFS	SML
短作业友好性	差	优	最优	优	优
交互响应性	差	中	最优	中	优
低优先级公平性	差	优	优	优	中
高优先级响应	中	差	最优	差	最优

5 实验总结

本次实验围绕进程调度算法展开，通过实现和比较多种调度算法，我取得了以下技术成果和理论认识：

5.1 掌握了调度框架的设计方法

我成功设计并实现了一个可扩展的调度框架，通过函数指针实现调度算法的动态切换。这种设计模式使得添加新的调度算法变得简单，只需实现统一接口并注册到函数指针数组即可。

5.2 深入理解了不同调度算法的特点

- **优先级调度**：采用最小堆实现，时间复杂度 $O(\log n)$ ；高优先级进程能快速响应，但可能导致低优先级进程饥饿
- **FCFS**：真正的非抢占式调度，进程运行直到主动让出 CPU；实现简单但可能导致护航效应
- **轮转调度**：抢占式调度，公平性好，适合交互式系统
- **静态多级队列**：结合了优先级和轮转的优点，在效率与公平之间取得平衡

5.3 掌握了数据结构优化技术

在优先级调度中，我使用最小堆替代了传统的多级队列或线性遍历方式。堆的上浮（siftUp）和下沉（siftDown）操作确保了 $O(\log n)$ 的时间复杂度，这是一个重要的算法优化实践。

5.4 掌握了进程统计信息收集方法

通过实现 `update_statistics()` 和 `wait2()` 系统调用，我学会了如何在内核中收集进程的运行时统计信息，这对于分析调度算法性能至关重要。

5.5 理解了调度决策对系统性能的影响

通过对比实验，我直观地观察到不同调度算法在响应时间、周转时间、公平性等方面的差异。这加深了我对“没有万能调度算法”这一理念的理解——选择合适的调度算法需要根据具体应用场景权衡。

6 实验心得

这次实验让我对操作系统调度有了更深刻的认识。

首先，我深刻体会到了调度算法的选择对系统行为的深远影响。在实现和测试不同调度算法的过程中，我发现看似简单的调度决策实际上涉及到响应时间、吞吐量、公平性等多个相互制约的因素。例如，优先级调度虽然能让高优先级进程快速响应，但如果不加限制，低优先级进程可能永远得不到执行机会，这就是所谓的“饥饿”问题。

其次，设计可扩展的调度框架让我对软件工程中的抽象和模块化有了新的理解。通过使用函数指针数组，我实现了调度算法的即插即用，这种设计使得添加新算法变得非常简单。这个经验让我意识到，好的架构设计能够极大地降低后续开发和维护的成本。

最后，通过收集和分析进程运行统计数据，我掌握了用数据说话的调试和分析方法。不同调度算法产生的统计数据差异直观地反映了它们的特性，这比单纯阅读理论知识更加印象深刻。

7 附录：核心代码实现

7.1 附录 A：数据结构定义

7.1.1 A.1 ptable.h - 进程信息结构

```
ptable.h

1 // ptable.h - 进程信息结构定义，用于内核与用户态程序之间
  传递进程信息
2 #ifndef _PTABLE_H_
3 #define _PTABLE_H_
4
5 #include "param.h" // 引入NPROC常量定义
6
7 // proc_info结构体：轻量级进程快照，专为用户态程序(如ps
  命令)设计
8 struct proc_info {
9     int pid; // 进程ID，唯一标识一个进
  程
10    int ppid; // 父进程ID，如果没有父进
  程则为-1
11    int priority; // 进程调度优先级(1-20，
  数值越小优先级越高)
12    int mem_size; // 进程占用的内存大小(字
  节数)
13    enum procstate state; // 进程状态
14    char name[16]; // 进程命令名(最多16字节)
15 };
16
17 #endif
```

7.1.2 A.2 proc.h - 进程控制块扩展

```

proc.h (关键部分)

// 进程状态枚举
enum procstate {
    UNUSED,      // 进程槽位未使用
    EMBRYO,      // 进程正在创建中
    SLEEPING,    // 进程因等待某个事件而睡眠
    RUNNABLE,    // 进程就绪，等待被调度执行
    RUNNING,     // 进程正在CPU上运行
    ZOMBIE       // 进程已终止，等待父进程回收
};

// 进程控制块(PCB)：内核中描述每个进程的核心数据结构
struct proc {
    uint sz;                      // 进程地址空间大小(字节)

    pde_t* pgdir;                // 进程页目录的虚拟地址
    char *kstack;                // 进程内核栈的底部地址
    enum procstate state;        // 进程当前状态
    int pid;                     // 进程ID
    int priority;                // 新增：进程优先级(1-20)

    struct proc *parent;         // 指向父进程的指针
    struct trapframe *tf;        // 保存用户态寄存器状态
    struct context *context;     // 进程上下文(用于上下文切换)

    void *chan;                  // 睡眠通道
    int killed;                  // 非零表示进程被标记为终止

    struct file *ofile[NOFILE]; // 打开的文件描述符数组
    struct inode *cwd;           // 当前工作目录
    char name[16];               // 进程名称
    int sem_held[32];            // 新增：记录进程持有的信号量资源
};

```

7.1.3 A.3 proc.c - 信号量结构定义

```
proc.c (信号量定义)

1      // 内核信号量结构：用于进程间的同步与互斥
2      struct semaphore {
3          int value;                // 信号量的当前值(可用资源数
   量)
4          int active;              // 标志位：1表示该信号量槽位
   已初始化
5          struct spinlock lock;    // 保护value和active字段的自
   旋锁
6      };
7
8      // 全局信号量表：系统最多支持32个信号量
9      struct semaphore sema[32];
```

7.2 附录 B：进程管理系统调用实现

7.2.1 B.1 proc.c - setpriority 函数

```
setpriority 实现

1      // setpriority: 修改指定进程的优先级
2      // 参数: pid - 目标进程ID, priority - 新优先级值(1-20)
3      // 返回值: 0表示成功, -1表示失败
4      int
5      setpriority(int pid, int priority)
6      {
7          struct proc *p;          // 用于遍历进程表的指针
8          int found = 0;           // 标志: 是否找到目标进程
9
10         // 步骤1: 参数校验 - 优先级必须在[1, 20]范围内
11         if(priority < 1 || priority > 20)
12             return -1;
13
14         // 步骤2: 获取进程表锁, 确保互斥访问
15         acquire(&ptable.lock);
```

```
setpriority 实现

16
17 // 步骤3: 遍历进程表, 查找目标进程
18 for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
19     if(p->pid == pid){
20         p->priority = priority; // 修改优先级
21         found = 1;
22         break;
23     }
24 }
25
26 // 步骤4: 释放进程表锁
27 release(&ptable.lock);
28
29 // 步骤5: 返回结果
30 return found ? 0 : -1;
31 }
```

7.2.2 B.2 proc.c - getptable 函数

```
getptable 实现

1 // getptable: 将内核进程表信息复制到用户空间
2 // 参数: ubuf - 用户空间缓冲区地址, size - 缓冲区大小
3 // 返回值: 0表示成功, -1表示失败
4 int
5 getptable(void *ubuf, int size)
6 {
7     struct proc *p; // 进程表遍历指针
8     struct proc_info pi; // 临时变量, 用于构造进
    程信息
9     char *uptr = (char*)ubuf; // 用户缓冲区指针
10    int i = 0; // 进程计数器
11
12    // 步骤1: 缓冲区大小检查
13    if(size < NPROC * sizeof(struct proc_info))
14        return -1;
```

```
getptable 实现

15
16 // 步骤2: 获取进程表锁
17 acquire(&ptable.lock);
18
19 // 步骤3: 遍历进程表
20 for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
21     // 清零临时结构体
22     memset(&pi, 0, sizeof(pi));
23
24     // 如果槽位正在使用, 填充信息
25     if(p->state != UNUSED){
26         pi.pid = p->pid;
27         pi.ppid = (p->parent) ? p->parent->pid : -1;
28         pi.priority = p->priority;
29         pi.mem_size = p->sz;
30         pi.state = p->state;
31         safestrcpy(pi.name, p->name, sizeof(pi.name)
);
32     }
33
34     // 步骤4: 复制到用户空间
35     if(copyout(myproc()->pgdir,
36 (uint)(uptr + i * sizeof(struct proc_info)),
37 (char*)&pi, sizeof(pi)) < 0){
38         release(&ptable.lock);
39         return -1;
40     }
41     i++;
42 }
43
44 // 步骤5: 释放锁并返回
45 release(&ptable.lock);
46 return 0;
47 }
```

7.2.3 B.3 sysproc.c - 系统调用包装器

● ● ● 系统调用包装器

```
1      // sys_setpriority: setpriority系统调用包装器
2      int
3      sys_setpriority(void)
4      {
5          int pid, priority;
6
7          // 从系统调用参数栈中解析参数
8          if(argint(0, &pid) < 0)          // 参数0: 进程ID
9              return -1;
10         if(argint(1, &priority) < 0)      // 参数1: 优先级值
11             return -1;
12
13         // 调用内核实现函数
14         return setpriority(pid, priority);
15     }
16
17     // sys_gettable: gettable系统调用包装器
18     int
19     sys_gettable(void)
20     {
21         char *buf;    // 用户空间缓冲区指针
22         int size;     // 缓冲区大小
23
24         // 解析参数并验证用户地址有效性
25         if(argptr(0, &buf, sizeof(struct proc_info) * NPROC)
26         ||
27         argint(1, &size))
28             return -1;
29
30         // 调用内核实现函数
31         return gettable(buf, size);
32     }
```

7.3 附录 C：用户态程序实现

7.3.1 C.1 ps.c - 进程查看命令

```
ps.c

1      #include "types.h"
2      #include "stat.h"
3      #include "user.h"
4      #include "param.h"    // 包含NPROC常量定义
5      #include "mmu.h"
6      #include "proc.h"     // 包含procstate枚举定义
7      #include "ptable.h"   // 包含proc_info结构定义
8
9      // 进程状态字符串数组：将状态枚举转换为可读字符串
10     static char *states[] = {
11         [UNUSED]    "UNUSED ", // 补空格对齐到8字符
12         [EMBRYO]    "EMBRYO ",
13         [SLEEPING]  "SLEEPING",
14         [RUNNABLE]  "RUNNABLE",
15         [RUNNING]   "RUNNING ",
16         [ZOMBIE]    "ZOMBIE "
17     };
18
19     int
20     main(int argc, char *argv[])
21     {
22         struct proc_info pinfo[NPROC]; // 接收进程信息的缓
23         冲区
24
25         // 步骤1：清零缓冲区，防止打印垃圾数据
26         memset(pinfo, 0, sizeof(pinfo));
27
28         // 步骤2：调用getptable系统调用获取进程表信息
29         if(getptable(pinfo, sizeof(pinfo)) < 0){
30             printf(2, "ps: getptable failed\n");
31             exit();
32         }
```

```
ps.c

33 // 步骤3: 打印表头
34 printf(1, "PID      PPID      PRI      MEM      STATE
CMD\n");
35
36 // 步骤4: 遍历并打印每个进程的信息
37 for(int i = 0; i < NPROC; i++){
38     // 跳过未使用的进程槽位
39     if(pinfo[i].pid == 0)
40         continue;
41
42     // 打印PID
43     printf(1, "%d      ", pinfo[i].pid);
44
45     // 打印PPID (-1则显示N/A)
46     if(pinfo[i].ppid == -1)
47         printf(1, "N/A      ");
48     else
49         printf(1, "%d      ", pinfo[i].ppid);
50
51     // 打印优先级和内存大小
52     printf(1, "%d      %d      ", pinfo[i].priority,
pinfo[i].mem_size);
53
54     // 打印进程状态
55     if(pinfo[i].state >= 0 && pinfo[i].state < 6)
56         printf(1, "%s      ", states[pinfo[i].state]);
57     else
58         printf(1, "???      ");
59
60     // 打印进程名称
61     printf(1, "%s\n", pinfo[i].name);
62 }
63
64 exit();
65 }
```

7.3.2 C.2 nice.c - 优先级修改命令

```
nice.c

1      #include "types.h"
2      #include "stat.h"
3      #include "user.h"
4      #include "fcntl.h"
5
6      int
7      main(int argc, char *argv[])
8      {
9          int pid, priority;
10
11         // 步骤1: 参数检查 - 需要两个参数: pid和priority
12         if(argc != 3){
13             printf(2, "Usage: nice pid priority\n");
14             exit();
15         }
16
17         // 步骤2: 解析命令行参数
18         pid = atoi(argv[1]);          // 第一个参数: 进程ID
19         priority = atoi(argv[2]);     // 第二个参数: 优先级值
20
21         // 步骤3: 参数校验 - 优先级必须在[1, 20]范围内
22         if (priority < 1 || priority > 20) {
23             printf(2, "nice: priority must be between 1 and
24             20\n");
25             exit();
26         }
27
28         // 步骤4: 调用setpriority系统调用
29         if(setpriority(pid, priority) < 0){
30             printf(2, "nice: failed to set priority (pid %d
31             not found?)\n", pid);
32         } else {
33             printf(1, "nice: priority of process %d set to %
34             d\n", pid, priority);
35         }
36     }
```

```
nice.c

33
34         exit();
35     }
```

7.4 附录 D：信号量实现

7.4.1 D.1 proc.c - sem_init 函数

```
sem_init 实现

1      // sem_init: 初始化一个信号量
2      // 参数: sem - 信号量ID(0-31), value - 信号量初始值
3      // 返回值: 0表示成功, -1表示失败
4      int
5      sem_init(int sem, int value)
6      {
7          // 步骤1: 参数校验
8          if(sem < 0 || sem >= 32)
9              return -1;
10
11         // 步骤2: 获取该信号量的锁
12         acquire(&sema[sem].lock);
13
14         // 步骤3: 检查是否已被初始化
15         if(sema[sem].active == 1) {
16             release(&sema[sem].lock);
17             return -1; // 已在使用, 不允许重复初始化
18         }
19
20         // 步骤4: 初始化信号量
21         sema[sem].value = value; // 设置初始值
22         sema[sem].active = 1;    // 标记为活跃状态
23
24         // 步骤5: 释放锁
25         release(&sema[sem].lock);
26     }
```

```
sem_init 实现

27         return 0;
28     }
```

7.4.2 D.2 proc.c - sem_destroy 函数

```
sem_destroy 实现

1      // sem_destroy: 销毁一个信号量
2      // 参数: sem - 信号量ID(0-31)
3      // 返回值: 0表示成功, -1表示失败
4      int
5      sem_destroy(int sem)
6      {
7          // 步骤1: 参数校验
8          if(sem < 0 || sem >= 32)
9              return -1;
10
11         // 步骤2: 获取锁
12         acquire(&sema[sem].lock);
13
14         // 步骤3: 检查是否处于活跃状态
15         if(sema[sem].active == 0) {
16             release(&sema[sem].lock);
17             return -1;
18         }
19
20         // 步骤4: 销毁信号量
21         sema[sem].active = 0; // 标记为未使用
22         sema[sem].value = 0; // 清零计数值
23
24         // 步骤5: 释放锁
25         release(&sema[sem].lock);
26
27         return 0;
28     }
```

7.4.3 D.3 proc.c - sem_wait 函数 (P 操作)

```
sem_wait 实现

1 // sem_wait: P操作, 申请资源, 资源不足则阻塞
2 // 参数: sem - 信号量ID, count - 需要申请的资源数量
3 // 返回值: 0表示成功, -1表示失败
4 int
5 sem_wait(int sem, int count)
6 {
7     // 步骤1: 参数校验
8     if(sem < 0 || sem >= 32)
9         return -1;
10
11     // 步骤2: 获取信号量锁
12     acquire(&sema[sem].lock);
13
14     // 步骤3: 检查是否处于活跃状态
15     if(sema[sem].active == 0){
16         release(&sema[sem].lock);
17         return -1;
18     }
19
20     // 步骤4: 循环检查资源是否足够 (核心逻辑)
21     // 使用while而非if, 防止虚假唤醒
22     while(sema[sem].value < count){
23         // 资源不足, 进程进入睡眠
24         // sleep的参数1: 睡眠通道 (使用信号量地址)
25         // sleep的参数2: 当前持有的锁
26         // sleep会原子性地释放锁并进入SLEEPING状态
27         sleep(&sema[sem], &sema[sem].lock);
28
29         // 被唤醒后, 回到while开头重新检查条件
30
31         // 防御性检查: 信号量是否在睡眠期间被销毁
32         if(sema[sem].active == 0){
33             release(&sema[sem].lock);
34             return -1;
35         }
36     }
```

```
sem_wait 实现

36     }
37
38     // 步骤5: 资源充足, 消耗资源
39     sema[sem].value -= count;
40
41     // 步骤6: 记账, 用于进程退出时的自动资源回收
42     myproc()->sem_held[sem] += count;
43
44     // 步骤7: 释放锁
45     release(&sema[sem].lock);
46
47     return 0;
48 }
```

7.4.4 D.4 proc.c - sem_signal 函数 (V 操作)

```
sem_signal 实现

1 // sem_signal: V操作, 释放资源并唤醒等待者
2 // 参数: sem - 信号量ID, count - 需要释放的资源数量
3 // 返回值: 0表示成功, -1表示失败
4 int
5 sem_signal(int sem, int count)
6 {
7     // 步骤1: 参数校验
8     if(sem < 0 || sem >= 32)
9         return -1;
10
11     // 步骤2: 获取信号量锁
12     acquire(&sema[sem].lock);
13
14     // 步骤3: 检查活跃状态
15     if(sema[sem].active == 0){
16         release(&sema[sem].lock);
17         return -1;
18     }
```

```
sem_signal 实现

19
20 // 步骤4: 释放资源, 增加信号量的值
21 sema[sem].value += count;
22
23 // 步骤5: 销账, 减少当前进程的资源持有计数
24 if(myproc()->sem_held[sem] >= count) {
25     myproc()->sem_held[sem] -= count;
26 } else {
27     // 防御性处理: 持有计数不足, 清零
28     myproc()->sem_held[sem] = 0;
29 }
30
31 // 步骤6: 唤醒所有等待该信号量的进程
32 // wakeup将所有在该通道上睡眠的进程状态改为RUNNABLE
33 wakeup(&sema[sem]);
34
35 // 步骤7: 释放锁
36 release(&sema[sem].lock);
37
38 return 0;
39 }
```

7.4.5 D.5 sysproc.c - 信号量系统调用包装器

```
信号量系统调用包装器

1 // sys_sem_init: sem_init系统调用包装器
2 int
3 sys_sem_init(void)
4 {
5     int sem, value;
6
7     if (argint(0, &sem) < 0) // 解析参数0: 信号量ID
8         return -1;
9     if (argint(1, &value) < 0) // 解析参数1: 初始值
10        return -1;
```

● ● ● 信号量系统调用包装器

```
11
12     return sem_init(&sem, 0, value);
13 }
14
15 // sys_sem_destroy: sem_destroy系统调用包装器
16 int
17 sys_sem_destroy(void)
18 {
19     int sem;
20
21     if (argint(0, &sem) < 0)
22         return -1;
23
24     return sem_destroy(&sem);
25 }
26
27 // sys_sem_wait: sem_wait系统调用包装器
28 int
29 sys_sem_wait(void)
30 {
31     int sem, count;
32
33     if (argint(0, &sem) < 0)           // 解析参数0: 信号量ID
34         return -1;
35     if (argint(1, &count) < 0)        // 解析参数1: 资源数量
36         return -1;
37
38     return sem_wait(&sem, count);
39 }
40
41 // sys_sem_signal: sem_signal系统调用包装器
42 int
43 sys_sem_signal(void)
44 {
45     int sem, count;
46 }
```

```
● ● ● 信号量系统调用包装器

47         if (argint(0, &sem) < 0)
48             return -1;
49         if (argint(1, &count) < 0)
50             return -1;
51
52         return sem_signal(sem, count);
53     }
```

7.5 附录 E：信号量测试程序

7.5.1 E.1 sem_test.c - 完整实现

```
● ● ● sem_test.c

1     #include "types.h"
2     #include "stat.h"
3     #include "user.h"
4     #include "fcntl.h"
5
6     // 测试参数定义
7     #define NUM_CHILDREN 10                // 并发子进程数量
8     #define TARGET_COUNT_PER_CHILD 50      // 每个子进程的循环
9     #define COUNTER_FILE "counter"         // 共享计数器文件名
10    #define SEM_ID 0                       // 使用0号信号量
11
12    // test_counter: 信号量互斥测试函数
13    void
14    test_counter()
15    {
16        int i, j;
17        int fd;
18        int val;
19
20        // ===== 第一部分：初始化 =====
21    }
```

```
sem_test.c

22 // 步骤1: 创建共享计数器文件并写入初始值0
23 fd = open(COUNTER_FILE, O_CREATE | O_RDWR);
24 if(fd < 0){
25     printf(1, "Error: create counter file failed\n")
;
26     exit();
27 }
28 val = 0;
29 write(fd, &val, sizeof(int)); // 写入4字节整数
30 close(fd);
31
32 // 步骤2: 初始化互斥信号量 (初始值为1, 作为互斥锁)
33 if(sem_init(SEM_ID, 1) < 0){
34     printf(1, "Error: sem_init failed\n");
35     exit();
36 }
37
38 printf(1, "Starting %d children, each incrementing %
d times...\n",
39 NUM_CHILDREN, TARGET_COUNT_PER_CHILD);
40
41 // ===== 第二部分: 创建并发子进程 =====
42
43 for(i = 0; i < NUM_CHILDREN; i++){
44     int pid = fork();
45     if(pid < 0){
46         printf(1, "fork failed\n");
47         exit();
48     }
49
50     if(pid == 0){
51         // --- 子进程代码 ---
52         for(j = 0; j < TARGET_COUNT_PER_CHILD; j++){
53
54             // P操作: 申请进入临界区
55             if(sem_wait(SEM_ID, 1) < 0){
```

```
sem_test.c

56         printf(1, "Child: sem_wait failed\n"
57     );
58         exit();
59     }
60     // ===== 临界区开始 =====
61
62     // 读取当前计数值
63     fd = open(COUNTER_FILE, O_RDONLY);
64     if(fd < 0){
65         printf(1, "Child: open read failed\n
66     ");
67         exit();
68     }
69     if(read(fd, &val, sizeof(int)) != sizeof
70     (int)){
71         printf(1, "Child: read failed\n");
72         exit();
73     }
74     close(fd);
75
76     // 更新计数值 (加1)
77     val++;
78
79     // 写回新值
80     fd = open(COUNTER_FILE, O_WRONLY);
81     if(fd < 0){
82         printf(1, "Child: open write failed\
83     n");
84         exit();
85     }
86     if(write(fd, &val, sizeof(int)) !=
87     sizeof(int)){
88         printf(1, "Child: write failed\n");
89         exit();
90     }
```

```
sem_test.c

87         close(fd);
88
89         // ===== 临界区结束 =====
90
91         // V操作：离开临界区
92         if(sem_signal(SEM_ID, 1) < 0){
93             printf(1, "Child: sem_signal failed\
n");
94             exit();
95         }
96     }
97
98     exit(); // 子进程完成后退出
99 }
100 }
101
102 // ===== 第三部分：等待子进程并验证结果 =====
103
104 // 父进程等待所有子进程退出
105 for(i = 0; i < NUM_CHILDREN; i++){
106     wait();
107 }
108
109 // 读取最终计数值
110 fd = open(COUNTER_FILE, O_RDONLY);
111 read(fd, &val, sizeof(int));
112 close(fd);
113
114 printf(1, "Final counter value: %d\n", val);
115
116 // 验证结果（期望值：10 × 50 = 500）
117 if(val == NUM_CHILDREN * TARGET_COUNT_PER_CHILD){
118     printf(1, "TEST PASSED!\n");
119 } else {
120     printf(1, "TEST FAILED! Expected %d\n",
NUM_CHILDREN * TARGET_COUNT_PER_CHILD);
121 }
```

```
sem_test.c

122     }
123
124     // ===== 第四部分：清理资源 =====
125
126     sem_destroy(SEM_ID);    // 销毁信号量
127     unlink(COUNTER_FILE);   // 删除测试文件
128
129     return;
130 }
131
132 int
133 main(int argc, char *argv[])
134 {
135     test_counter();
136     return 0;
137 }
```